CAPSTONE PROJECT PORTFOLIO

Python, Django and FastAPI

Job & Employability Progression Management System

Portfolio details	Sample entry
Learner name	[Insert learner name]
Programme	Software Development – Python Full Stack
Project title	Job & Employability Progression Management System
Project version	1.0
Submission date	[Insert date]
GitHub repository	[Insert actual repository URL]
Live application	[Insert deployed application URL, if available]

Project purpose

This portfolio documents the planning, design, development, testing, security review, documentation and deployment of a web-based job management application. The system provides separate functions for job seekers, employers and administrators.

Submission note

This is a worked sample portfolio. Replace every square-bracket placeholder with genuine learner details and attach the actual project files, repository evidence, screenshots, test outputs and deployment records before submission. Sample test outcomes and example records are illustrative and must not be presented as completed work unless they have been verified.

Portfolio contents

Section	Description
1	Project overview and objectives
2	Requirements and scope
3	User stories and acceptance criteria
4	Project planning and task management
5	System design and wireframes
6	Database design and Django models
7	Development implementation
8	GitHub and version control evidence
9	Testing strategy and test evidence
10	Defect management
11	Security review
12	AI development journal
13	User guide
14	README and technical documentation
15	Deployment and release evidence
16	Final review and evidence checklist

Evidence convention

Where this document says **Evidence to insert**, include the relevant screenshot, source file extract, exported report or working link. Give each item a clear filename and date. Do not use a generic image as proof of the learner's own implementation.

1. Project overview and objectives

Project title: Job & Employability Progression Management System

Project summary

The proposed system is a web application designed to support the management of job vacancies and job applications. Employers can create and manage vacancies and review applications. Job seekers can create an account, browse available vacancies, submit applications and monitor application progress. An administrator can oversee user and vacancy records.

Problem being addressed

Job seekers may use separate websites, spreadsheets or notes to keep track of vacancies and applications. Employers may receive applications through different channels, making it harder to maintain consistent records. A central application can make these activities easier to organise and review.

Project objectives

- Provide registration and login for the relevant user roles.
- Allow employers to create, update, publish and close vacancies.
- Allow job seekers to browse and search vacancies.
- Allow job seekers to submit applications and view their status.
- Allow employers to review applications for their vacancies.
- Validate user input and protect restricted pages.
- Provide automated tests for important application behaviour.
- Document installation, use, testing and deployment.

Proposed technology stack

Area	Technology	Purpose
Backend web framework	Django	User accounts, server-rendered pages, business logic and ORM.
API framework	FastAPI	Selected API endpoints for structured data access or integration.
Programming language	Python	Application logic and automated tests.
Database	SQLite for development	Store users, employers, vacancies and applications. A production database may differ.
Frontend	HTML, CSS, Bootstrap, JavaScript	Responsive user interface and client-side interaction.
Version control	Git and GitHub	Source control, issues, project planning and collaboration.
Testing	Django TestCase / pytest	Check expected behaviour and API responses.

2. Requirements and scope

Functional requirements

ID	Requirement	Priority
FR01	A visitor can register as a job seeker or employer.	High
FR02	A registered user can log in and log out.	High
FR03	An employer can create a vacancy.	High
FR04	An employer can edit or close a vacancy they own.	High
FR05	A job seeker can view and search published vacancies.	High
FR06	A job seeker can apply to a vacancy.	High
FR07	A job seeker can view their submitted applications and statuses.	High
FR08	An employer can view applications for their vacancies.	High
FR09	An authorised user can update an application status.	Medium
FR10	An administrator can manage system records.	Medium

Non-functional requirements

- Usability: navigation and forms should be clear and consistent.
- Security: authenticated pages and role-specific actions must be protected.
- Reliability: invalid input should be rejected with a useful validation message.
- Maintainability: code should be organised into logical Django apps and reusable components.
- Compatibility: the interface should work on current desktop and mobile browsers.
- Performance: common list and detail pages should respond within an acceptable time for the expected sample dataset.

Project scope

Included: account access, employer profiles, vacancy management, vacancy browsing, job applications, application status tracking, validation, testing and documentation. Excluded from the initial release: payment processing, automated CV scoring, external job-board synchronisation, interview video calls and a production-scale notification service.

Assumptions and constraints

The initial development environment uses Python and SQLite. Email delivery may be represented by a development console backend unless a real email service is configured. The project is an educational application and should not store real sensitive candidate information during demonstration.

3. User stories and acceptance criteria

User stories describe the system from the perspective of the person using it. The acceptance criteria define conditions that can be checked during development and testing.

ID	Role	User story
US01	Job seeker	As a job seeker, I want to register so that I can use the platform.
US02	Employer	As an employer, I want to create a vacancy so that candidates can apply.
US03	Job seeker	As a job seeker, I want to search vacancies so that I can find relevant opportunities.
US04	Job seeker	As a job seeker, I want to submit an application so that an employer can review it.
US05	Job seeker	As a job seeker, I want to track application status so that I know its progress.
US06	Employer	As an employer, I want to view applications so that I can assess candidates.
US07	Employer	As an employer, I want to update a vacancy so that its details remain accurate.

Acceptance criteria – US02: Create a vacancy

Given	When	Then
The employer is logged in and authorised	They submit a form with valid required fields	The vacancy is saved and appears in the employer's vacancy list.
The employer is logged in	They submit the form without a job title	The form displays a validation message and no vacancy is created.
The employer is logged in	They enter an invalid salary value	The form rejects the value and explains the validation issue.
A user is not logged in	They request the vacancy creation page	They are redirected to the login page or shown an access-denied response.

Acceptance criteria – US04: Apply for a vacancy

Given	When	Then
A job seeker is logged in	They submit a valid application	The application is stored and linked to the selected vacancy and applicant.
A job seeker is logged in	They submit an incomplete application	Validation messages are displayed.
A job seeker has already applied	They attempt to apply again to the same vacancy	The system prevents a duplicate application.
A vacancy is closed	A job seeker attempts to apply	The application is rejected and the user is informed.

Traceability: Map each user story to one or more development tasks and test cases. For example, US02 should link to the vacancy form, create-vacancy view, model validation and test cases for valid and invalid submissions.

4. Project planning and task management

Indicative eight-week plan

Week	Activities	Output / evidence
1	Confirm problem, users, requirements and project scope.	Proposal, requirements list and user stories.
2	Create wireframes, data model and architecture plan.	Wireframes, ER diagram and design notes.
3	Set up repository, Django project, database and authentication.	Initial commits, project structure and login tests.
4	Develop employer profile and vacancy management.	Vacancy CRUD pages, model and tests.
5	Develop vacancy search and job application workflow.	Application pages, status handling and tests.
6	Develop selected FastAPI endpoints and integration if required.	API routes, schemas, API tests and documentation.
7	Run functional, validation and security checks; fix defects.	Test report, defect log and security checklist.
8	Complete user guide, README, release notes and deployment.	Final documentation and deployment evidence.

Example task breakdown

Task ID	Task	Priority	Dependency
T01	Create GitHub repository and README	High	None
T02	Create Django project and app structure	High	T01
T03	Create user role and employer models	High	T02
T04	Implement login and access restrictions	High	T03
T05	Implement vacancy create, read, update and close	High	T03, T04
T06	Implement vacancy search and detail pages	High	T05
T07	Implement job application workflow	High	T05, T06
T08	Create FastAPI endpoints and schemas	Medium	T03
T09	Write and run automated tests	High	T04–T08
T10	Resolve defects and review security	High	T09
T11	Prepare user guide and deployment	High	T10

GitHub Project board workflow

Create a GitHub Project board with the columns **Backlog**, **Ready**, **In Progress**, **Review** and **Done**. Add one issue for each task or user story, set priorities, assign owners where relevant and record dependencies in issue descriptions. Move cards as work progresses.

Evidence to insert

- Screenshot of the actual GitHub Project board showing workflow columns and task cards.
- Screenshots or exports of representative GitHub Issues.
- A dated project plan showing planned and actual progress.

5. System design and wireframes

Proposed user journeys

Job seeker: Register → Log in → Browse vacancies → View vacancy details → Apply → View application status.

Employer: Register → Log in → Complete employer profile → Create vacancy → Review applications → Update application status.

Administrator: Log in → Review users and vacancies → Manage records and respond to reported issues.

Suggested screen list

Screen	Main elements	Access
Registration	Name, email, password, account type and validation messages.	Public
Login	Email/username, password and error feedback.	Public
Job seeker dashboard	Profile summary, recent applications and application statuses.	Job seeker
Employer dashboard	Vacancy count, active vacancies and recent applications.	Employer
Vacancy list	Search field, filters, vacancy cards and pagination.	Public / logged-in
Vacancy detail	Title, employer, description, location, salary and apply action.	Public / logged-in
Vacancy form	Title, description, location, salary, closing date and status.	Employer
Applications	Candidate details, vacancy, submission date and status.	Employer / job seeker

Evidence to insert

Insert the learner's own wireframe images for the registration page, login page, job seeker dashboard, employer dashboard, vacancy listing, vacancy detail, vacancy form and application management page. Label each wireframe with its screen name and version.

Architecture overview

The browser sends requests to the Django application. Django views process requests, apply access checks and use Django models to read or write database records. Templates render the user interface. A FastAPI service may expose selected API endpoints using Pydantic schemas and service logic. The API should use clearly defined request and response formats.

> **Evidence to insert:** A system architecture diagram showing browser, Django application, FastAPI service (if implemented), database and any external services actually used.

6. Database design and Django models

Core entities

Entity	Example fields	Purpose
User	id, username, email, password hash, role	Authentication and role identification.
Employer	id, user_id, company_name, contact_email	Employer organisation details.
Vacancy	id, employer_id, title, description, location, salary, status, created_at	Vacancy information owned by an employer.
JobApplication	id, vacancy_id, applicant_id, cover_message, status, submitted_at	Application submitted by a job seeker.

Relationship summary

- One employer can publish many vacancies.
- One vacancy can receive many applications.
- One job seeker can submit many applications.
- Each application belongs to one vacancy and one applicant.

Illustrative Django model code

```
from django.conf import settings from django.db import models class Employer(models.Model): user = models.OneToOneField( settings.AUTH_USER_MODEL, on_delete=models.CASCADE ) company_name = models.CharField(max_length=200) contact_email = models.EmailField() class Vacancy(models.Model): employer = models.ForeignKey( Employer, on_delete=models.CASCADE, related_name="vacancies" ) title = models.CharField(max_length=200) description = models.TextField() location = models.CharField(max_length=150) salary = models.DecimalField( max_digits=10, decimal_places=2, null=True, blank=True ) status = models.CharField( max_length=20, default="open" ) class JobApplication(models.Model): vacancy = models.ForeignKey( Vacancy, on_delete=models.CASCADE, related_name="applications" ) applicant = models.ForeignKey( settings.AUTH_USER_MODEL, on_delete=models.CASCADE ) cover_message = models.TextField(blank=True) status = models.CharField( max_length=30, default="submitted" ) submitted_at = models.DateTimeField(auto_now_add=True) class Meta: constraints = [ models.UniqueConstraint( fields=["vacancy", "applicant"], name="unique_applicant_per_vacancy" ) ]
```

This is an example model design. Adapt field names, user-role handling and validation to the actual implementation. Run migrations and tests against the completed project.

Evidence to insert

- ER diagram exported as PNG or PDF.
- Actual Django models.py file or repository link.
- Migration files and evidence that migrations were applied successfully.

7. Development implementation

Suggested Django project structure

```
job_management/ ■■■■ manage.py ■■■■ config/ ■ ■■■■ settings.py ■ ■■■■ urls.py ■ ■■■■ asgi.py ■■■■
accounts/ ■ ■■■■ models.py ■ ■■■■ forms.py ■ ■■■■ views.py ■ ■■■■ urls.py ■■■■ employers/ ■ ■■■■
models.py ■ ■■■■ forms.py ■ ■■■■ views.py ■ ■■■■ urls.py ■■■■ vacancies/ ■ ■■■■ models.py ■ ■■■■
forms.py ■ ■■■■ views.py ■ ■■■■ urls.py ■■■■ applications/ ■ ■■■■ models.py ■ ■■■■ views.py ■ ■■■■
urls.py ■■■■ templates/ ■■■■ static/ ■■■■ requirements.txt
```

Development approach

- Create the project and apps using Django conventions.
- Configure the database and create models for the main entities.
- Create and apply migrations.
- Implement forms and server-side validation.
- Use authentication and permission checks for restricted views.
- Build templates using reusable base layouts and static assets.
- Implement CRUD operations and application status handling.
- Add API endpoints in FastAPI only where the project requires them.
- Write automated tests alongside the implementation.

FastAPI example – illustrative endpoint

```
from fastapi import FastAPI from pydantic import BaseModel app = FastAPI(title="Job Management API")
class VacancySummary(BaseModel): id: int title: str location: str @app.get("/api/vacancies",
response_model=list[VacancySummary]) def list_vacancies(): # Replace with a database/service-layer
query. return []
```

The example endpoint returns an empty list until connected to a data source. The final portfolio should include the actual API implementation, response examples and test results if FastAPI is part of the submitted project.

Evidence to insert

- Repository link and a short explanation of the main folders.
- Selected source code extracts with filenames.
- Screenshots of the working application.
- API documentation screenshot or OpenAPI URL, if FastAPI is implemented.

8. GitHub and version control evidence

Repository details

Item	Learner entry
Repository name	[Insert repository name]
Repository URL	[Insert GitHub URL]
Default branch	[e.g. main]
Project board URL	[Insert GitHub Project URL]
Development period	[Start date] to [End date]

Version control practice

Use meaningful commit messages that describe the change. Commit related work in manageable units. Keep passwords, API keys, database credentials and other secrets out of the repository. Use branches where appropriate and review changes before merging.

Example commit message formats

- Initial Django project structure
- Add employer and vacancy models
- Implement vacancy creation form
- Prevent duplicate applications
- Add tests for vacancy permissions
- Update README installation instructions

Evidence to insert

- Repository home page showing project files.
- Commit history showing development over time.
- Issues showing tasks linked to requirements or user stories.
- Project board showing task status.
- Pull requests or code review evidence, if used.

9. Testing strategy and test evidence

Testing objectives

Testing checks whether the application meets its requirements, handles invalid input safely and prevents unauthorised actions. Tests should be repeatable and linked to user stories or functional requirements.

Test levels

Test type	Purpose	Example
Unit test	Check a small function or model rule.	Check vacancy salary validation.
View / integration test	Check a request and its interaction with application components.	Submit a valid vacancy form and verify the saved record.
API test	Check endpoint status codes and response data.	GET /api/vacancies returns the expected structure.
System test	Check a complete user journey.	Register, log in, apply for a vacancy and view status.
Security test	Check access restrictions and input handling.	A job seeker cannot edit another employer's vacancy.

Sample test cases

Test ID	Scenario	Expected result	Actual result
TC01	Register with valid details	Account is created.	[Record result]
TC02	Register with an existing email	Duplicate account is rejected.	[Record result]
TC03	Log in with an incorrect password	Access is denied and an error is shown.	[Record result]
TC04	Employer creates a valid vacancy	Vacancy is saved and displayed.	[Record result]
TC05	Vacancy form submitted without title	Validation error is displayed.	[Record result]
TC06	Job seeker applies to an open vacancy	Application is saved.	[Record result]
TC07	Job seeker applies twice to same vacancy	Duplicate application is prevented.	[Record result]
TC08	Job seeker attempts employer-only action	Access is denied.	[Record result]
TC09	Employer views applications for own vacancy	Relevant applications are displayed.	[Record result]
TC10	Employer attempts to edit another employer's vacancy	Access is denied.	[Record result]

Do not mark the sample cases as Pass until each test has been executed. Record the date, environment, actual result, pass/fail outcome and any defect reference.

Automated test example

```
from django.test import TestCase from django.urls import reverse class VacancyAccessTests(TestCase): def test_vacancy_create_requires_login(self): response = self.client.get( reverse("vacancy_create") ) self.assertIn(response.status_code, [302, 401, 403])
```

Adapt the URL name and expected response to the real application. Include the full test file and the actual test-run output in the submitted evidence.

10. Defect management

Record each defect when it is found. Include enough detail for another developer to reproduce the issue and confirm the fix.

Defect ID	Example issue	Severity	Status / evidence
BUG01	Vacancy form accepts a missing job title.	Medium	[Open / Fixed – add evidence]
BUG02	Job seeker can access an employer-only page.	High	[Open / Fixed – add evidence]
BUG03	Duplicate application can be submitted.	High	[Open / Fixed – add evidence]
BUG04	Vacancy list does not show a useful empty state.	Low	[Open / Fixed – add evidence]

Defect record template

Field	Details to record
Defect ID	Unique reference, e.g. BUG05
Date found	[Date]
Environment	Browser, operating system, application version
Steps to reproduce	Numbered steps from the starting page
Expected result	What should happen
Actual result	What happened
Severity / priority	Critical, High, Medium or Low
Resolution	Code or configuration change made
Retest result	Pass / Fail with test evidence

11. Security review

The following checklist provides a starting point for reviewing the application. Mark each item only after checking the implementation.

Check	Review action	Result
Authentication	Use the framework's password hashing and authentication features.	[Not checked / Pass / Fail]
Authorisation	Check that each role can access only permitted functions and records.	[Not checked / Pass / Fail]
CSRF	Keep CSRF protection enabled for relevant form submissions.	[Not checked / Pass / Fail]
Input validation	Validate data on the server; do not rely only on browser validation.	[Not checked / Pass / Fail]
ORM / SQL	Use Django ORM or parameterised queries; avoid unsafe SQL concatenation.	[Not checked / Pass / Fail]
Secrets	Read secrets from environment variables or a secure secret store.	[Not checked / Pass / Fail]
Debug settings	Disable DEBUG in production and configure allowed hosts.	[Not checked / Pass / Fail]
HTTPS	Use HTTPS for the deployed application.	[Not checked / Pass / Fail]
File uploads	Validate file type and size if uploads are supported.	[Not checked / Pass / Fail]
Dependencies	Review and update project dependencies where appropriate.	[Not checked / Pass / Fail]

Security notes

The final submission should explain any security limitations that remain. Never include real passwords, secret keys, private candidate information or unredacted credentials in screenshots or repository files. A checklist is evidence of review only when it is completed against the actual application.

12. AI development journal

If AI tools were used, record how they supported the work, how suggestions were checked and what changes were made. The journal should be an honest record of the learner's own use of AI.

Date	Task / issue	AI prompt or assistance	Verification and outcome
[Date]	[e.g. understand a Django form validation error]	[Paste the actual prompt or summarise the assistance]	[Explain what was tested and whether the suggestion was used, changed or rejected.]
[Date]	[e.g. review API response structure]	[Record actual prompt]	[Record tests or documentation checks.]
[Date]	[e.g. improve README clarity]	[Record actual prompt]	[Explain what was edited and checked.]

Reflection prompts

- What did the AI tool help you understand or complete?
- What parts of the suggestion did you verify independently?
- Did you reject or modify any suggestion? Why?
- What did you learn from checking the result?
- What steps did you take to avoid exposing personal or confidential data?

If no AI tools were used, state that clearly and explain that the journal is not applicable to the project.

13. User guide

13.1 Job seeker – registration and login

1. Open the application using the supplied URL.
2. Select the registration option.
3. Enter the required details and submit the form.
4. Use the registered credentials to log in.
5. If validation messages appear, correct the information and submit again.

13.2 Search for vacancies

1. Open the vacancy listing page.
2. Enter a keyword or select an available filter.
3. Review the vacancy title, employer and location.
4. Open a vacancy to read its full description and requirements.

13.3 Submit and track an application

1. Log in as a job seeker.
2. Open an available vacancy.
3. Select the apply option and complete the application form.
4. Submit the form and review the confirmation message.
5. Open the applications page to view the current status.

13.4 Employer – manage vacancies

1. Log in using an employer account.
2. Open the employer dashboard.
3. Select the option to add a vacancy.
4. Complete the required fields and submit.
5. Open the vacancy list to edit or close a vacancy.

13.5 Employer – review applications

1. Open the employer dashboard or vacancy list.
2. Select a vacancy to view its applications.
3. Review the submitted application details.
4. Update the application status where the system permits.
5. Save the change and confirm the updated status.

Support and known limitations

[Insert the actual support contact or project owner details.]
[List known limitations, supported browsers, test account instructions and any features not included in the release.]

> **Evidence to insert:** Add numbered screenshots that match the steps above. Use screenshots from the learner's working application.

14. README and technical documentation

README.md – recommended content

```
# Job & Employability Progression Management System ## Project overview A web application for managing job vacancies and applications. ## Features - User registration and login - Employer vacancy management - Vacancy browsing and search - Job application submission - Application status tracking ## Technology - Python - Django - FastAPI (if implemented) - SQLite (development) - HTML, CSS, Bootstrap and JavaScript ## Installation 1. Clone the repository. 2. Create and activate a virtual environment. 3. Install dependencies from requirements.txt. 4. Configure environment variables. 5. Run database migrations. 6. Create an administrator account if required. 7. Start the development server. ## Run the application python manage.py runserver ## Testing [Add the actual automated test command and test results.] ## API documentation [Add the actual API documentation URL, if available.] ## Screenshots [Add screenshots of the implemented application.] ## Deployment [Add the live application URL, if deployed.]
```

Technical documentation should also explain

- Project setup and dependency installation.
- Environment configuration and database setup.
- Application folders and the purpose of each Django app.
- Important models, views, forms, URLs and templates.
- FastAPI routers, schemas and endpoints, if used.
- How to run automated tests.
- How to troubleshoot common setup or runtime errors.

Release notes template

Version	Date	Changes	Known issues
1.0	[Date]	[List features included in this release]	[List any known issues]

15. Deployment and release evidence

Deployment record

Item	Learner entry
Hosting provider	[Insert provider]
Application URL	[Insert live URL]
Deployment date	[Insert date]
Python version	[Insert version]
Database used	[Insert database]
Static files configuration	[Describe actual configuration]
Environment configuration	[Describe without revealing secrets]
Deployment status	[Successful / In progress / Not deployed]

Deployment verification checklist

- Application opens at the supplied URL.
- Login and logout work on the deployed version.
- Role-based pages are protected.
- Vacancy listing and detail pages load correctly.
- Application submission works and data is saved.
- Static files and styling load correctly.
- Production debug settings and allowed hosts are configured.
- Secrets are not committed to the repository.
- Database and backup arrangements are understood.

Evidence to insert

- Screenshot of the live application homepage.
- Screenshot of a successful login and key workflow.
- Deployment configuration with secrets hidden.
- A record of deployment checks and any issues found.

If the application has not been deployed, state this clearly and provide the local run instructions and evidence of successful local testing. Do not invent a live URL.

16. Final review and evidence checklist

Complete this checklist before submission. Use the status column to record whether the evidence is included and verified.

No.	Portfolio evidence	Status / reference
1	Requirements and project scope	[Included / reference]
2	User stories	[Included / reference]
3	Acceptance criteria	[Included / reference]
4	Project plan and planning evidence	[Included / reference]
5	Wireframes	[Included / reference]
6	Database / model design	[Included / reference]
7	Source code	[Repository / folder]
8	GitHub repository and commit history	[URL / screenshot]
9	GitHub Issues and project board	[URL / screenshot]
10	Application screenshots	[Folder / filenames]
11	Test plan and test cases	[File reference]
12	Automated tests and test results	[File / output reference]
13	Defect log	[File reference]
14	Security checklist	[File reference]
15	AI development journal	[File reference / N/A]
16	README	[Repository path]
17	User guide	[File reference]
18	Deployment evidence	[URL / screenshots / not deployed]

Project reflection

What went well?
[Describe the parts of the project that were completed successfully and provide examples.]

What challenges were encountered?
[Describe technical or planning challenges and how they were investigated.]

What would be improved in a future version?
[Identify realistic improvements and explain why they would be valuable.]

What skills were developed?
[Reflect on programming, database, testing, version control, communication and problem-solving skills.]

Declaration

I confirm that this portfolio represents my own project work. Any external resources or AI assistance used have been acknowledged where required. The evidence included reflects the work completed and has not been fabricated.

Learner name	[Insert name]
Signature	[Sign here]
Date	[Insert date]

End of portfolio